

一种基于 LLM 辅助的 Tokio 异步状态机动态跟踪分析

从 GDB 原始日志到可解释报告

陈玺州

2026 年春夏开源操作系统训练营

问题意识

LLM 辅助异步
状态机跟踪

陈玺州

研究动机
问题意识
技术路线

实验采集

LLM 分析

控制块视角

状态机结论

方法反思

目标

不只让异步爬虫能运行，而是解释 `async/await` 在运行时如何保存状态、返回 Pending、被 Waker 唤醒并继续执行。

- 表层代码只是几个 `.await`
- 底层行为涉及 future frame、寄存器、栈帧、捕获变量区和 runtime 调度
- 核心挑战：把高层抽象还原为可观察的机器状态

整体技术路径

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

问题意识

技术路线

实验采集

LLM 分析

控制块视角

状态机结论

方法反思



核心优点

暴露 LLM 得出结果的路径：每个结论都可以回溯到命令、断点、日志、寄存器、内存地址和调用栈。

第一步：获得原始数据

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

原始数据

断点设计

LLM 分析

控制块视角

状态机结论

方法反思

```
cargo build --bin async
timeout 35s rust-gdb -q target/debug/async -x gdb_scripts/
gdb_track.gdb
```

- 使用 dev profile, 保留 debuginfo
- 先限制 SPIDER_CONCURRENCY=1, 观察单个状态机的完整生命周期
- 再追加 SPIDER_CONCURRENCY=4, 观察多个状态机实例并存
- 原始输出保存为 GDB 日志, 再整理为中文分析版

断点设计

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

原始数据

断点设计

LLM 分析

控制块视角

状态机结论

方法反思

位置	观察目标
download_async.rs:16	外层 create_dir_all.await
download_async.rs:25	每个 task 的 async block
downloader_async.rs:13--14	request send().await
downloader_async.rs:16--17	response.bytes().await
downloader_async.rs:20	内部 create_dir_all(parent).await
downloader_async.rs:23	tokio::fs::write(output, bytes).await

设计原则

断点尽量卡在 await 前后，让日志呈现 “保存状态 -> Pending -> 唤醒 -> 恢复” 的切换过程。

第二步：AI 辅助分析

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

分析流程

证据链

人工校验

控制块视角

状态机结论

方法反思

结构化

按源码位置、状态机层级、寄存器和 backtrace 归类日志。

生成假设

基于 ABI、`Future::poll` 形态和内存内容推断地址含义。

证据对照

要求每个结论说明来自哪条日志、哪个地址或哪个变量。

使用原则

LLM 不直接给最终裁决，而是帮助整理证据链、发现重复模式、标注不确定性。

证据链示例

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

分析流程

证据链

人工校验

控制块视角

状态机结论

方法反思

观察值	解释依据
rdx = 0xc	对应”output/async” 的 12 字节长度 小端序能读出”ut/async” 片段
rcx =	
0x636e...7475	附近内存包含”output/async” backtrace 中多次作为外层 cx 出现 per-task 捕获区，包含 task/url/output
0x555556351ce0	
0x7fffffffaff8	
0x555556355d10	

关键点

不是 “AI 说它是什么”，而是 “日志中的哪些事实支持这个判断”。

第三步：人工提炼与校验

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

分析流程

证据链

人工校验

控制块视角

状态机结论

方法反思

- 稳定事实：反复出现、能和 backtrace 或内存内容相互印证
- 临时现象：源码行断点处的编译器临时值，不强行解释
- 结论边界：明确哪些地址能对应变量，哪些寄存器不能稳定解读

底层分析里的危险

LLM 容易把所有值都解释完整；但系统调试需要保留“不知道”，否则会把临时值包装成错误结论。

从状态机到控制块

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

从状态机到控制块
控制块字段

Waker 与唤醒
与 OS TCB 对比

状态机结论

方法反思

编译器生成的部分

- Future frame
- 状态编号：
Unresumed/SuspendN/完成
- 跨 await 仍需使用的局部变量
- 当前正在等待的子 future：
__awaiter

运行时管理的部分

- 是否 ready / running / complete
- Waker 和 ready queue 节点
- 任务归属的 executor / scheduler
- 完成结果和等待者

控制块视角

Future frame 解释“从哪里继续执行”；任务控制元数据解释“谁在什么时候再次 poll 它”。

协程任务控制块包含什么

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

从状态机到控制块

控制块字段

Waker 与唤醒

与 OS TCB 对比

状态机结论

方法反思

字段类别	作用
state bits	记录 running / notified / complete / cancelled 等生命周期状态
future frame ptr waker	指向保存 async 状态机和跨 await 变量的对象 I/O ready 后重新找到这个任务
queue links	挂到 ready queue 或 FuturesUnordered 的内部队列
output / join scheduler handle	任务完成后的返回值、错误和等待者关系 知道应该交给哪个 runtime / executor 继续调度

本实验里的对应关系

每个下载并不是显式 `tokio::spawn` 出来的 `JoinHandle`；更直接的调度单位是 `buffer_unordered / FuturesUnordered` 内部保存的子 future 节点。

Waker: 控制块的回指针

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

从状态机到控制块

控制块字段

Waker 与唤醒

与 OS TCB 对比

状态机结论

方法反思

```
RawWaker {  
    data: *const (), // 指向 task node / 控制块  
    vtable: *const RawWakerVTable  
}
```

```
RawWakerVTable:  
    clone -> clone_arc_raw  
    wake -> wake_arc_raw  
    wake_by_ref -> wake_by_ref_arc_raw  
    drop -> drop_arc_raw
```

- 子 future 返回 Pending 时, 通过 Context 注册 Waker
- I/O ready 后调用 wake_by_ref, 把对应节点重新标记为 ready
- 下一轮 FuturesUnordered::poll_next 选择 ready 子 future 继续 poll

与操作系统 TCB 的对照

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

从状态机到控制块

控制块字段

Waker 与唤醒

与 OS TCB 对比

状态机结论

方法反思

维度	OS thread TCB	Tokio / Future 任务节点
调度单位	内核线程	用户态 future 实例
保存现场	寄存器 + 独立线程栈	future frame + 状态位
等待 I/O	阻塞线程, 内核切换	返回 Pending, 注册 Waker
恢复方式	调度回同一线程上下文	executor 再次 poll
局部变量	主要在线程栈中	跨 await 的变量进入 frame

关键差异

Tokio task 不保存完整 CPU 上下文和私有 OS 栈; 一次 poll 临时借用 worker 栈, 持久状态保存在 future frame 和任务元数据中。

核心结论：嵌套状态机

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

嵌套结构

单并发

并发 4

方法反思

外层下载调度状态机

- > collect / buffer_unordered / FuturesUnordered
- > per-task async move 状态机
- > CrawlTask::run_async_to 状态机
- > downloader_async 状态机
- > request send / bytes / tokio fs 子状态机

- 每个 await 把后续要用的局部变量保存到 future frame
- 子 future Pending 后，控制权返回 runtime
- I/O ready 后，Waker 触发 runtime 再次 poll
- 状态机跳回 await 后的源码位置继续执行

单并发观察

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

嵌套结构

单并发

并发 4

方法反思

- line 16 → line 17: 外层 `create_dir_all.await` 返回
- line 25 反复命中: 同一个 per-task future 被多次 poll
- line 13–14 → line 15: `send().await` 返回 Response
- line 16–17 → line 19: `bytes().await` 返回 Bytes
- line 23: `write(output, bytes).await` 使用已保存的状态

结论

单并发适合观察一个状态机的保存、Pending、唤醒和恢复。

并发 4 观察

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

嵌套结构

单并发

并发 4

方法反思

任务	捕获变量区地址
北京大学	0x555556355d10
清华大学	0x555556353f10
中国人民大学	0x555556353b00
北京师范大学	0x555556340590

- 同一种 async block 类型可以同时有多个实例
- 后续日志中 rsi 在四个地址之间轮换
- FuturesUnordered 保存子 future，并借助 waker/ready queue 调度

可解释性来自哪里

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

方法反思

可解释性

LLM 使用

总结

- 1 命令可复现：构建命令、GDB 脚本、并发参数明确
- 2 证据可追踪：结论能回到断点、寄存器、内存和 backtrace
- 3 LLM 路径可暴露：结构化、假设、证据对照、人工校验分层清楚
- 4 不确定性可保留：不能稳定对应的值不写成确定结论

如何更好地使用 LLM

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

方法反思

可解释性

LLM 使用

总结

- 先给原始数据，再要求分析
- 把大问题拆成可验证的小问题
- 要求输出证据链，而不是只输出结论
- 主动标注不确定性和反例
- 最终报告由人收束，保证问题意识和结论边界

一句话

LLM 的价值不是替代推理，而是把推理过程显性化、结构化、可复查。

总结

LLM 辅助异步
状态机跟踪

陈玺州

研究动机

实验采集

LLM 分析

控制块视角

状态机结论

方法反思

可解释性

LLM 使用

总结

技术收获

看清了 Tokio 异步程序中“一堆状态机”在内存和调度上的实际表现。

方法收获

建立了从原始日志到 LLM 辅助分析，再到人工提炼报告的可解释工作流。

训练营后的理解

系统软件里真正有价值的结论，不是听起来合理，而是能被日志、代码和机器状态共同支撑。